K-Nearest Neighbors Classification

Nakul D. M.[1], Smriti Kaul[1], Rishabh R.[1],
[1]*Department of Computer Science and Engineering,*
Amrita School of Computing, Amrita Vishwa Vidyapeetham, Bengaluru, India
bl.sc.u4cse24132@bl.students.amrita.edu, bl.sc.u4cse24149@bl.students.amrita.edu, bl.sc.u4cse24143@bl.students.amrita.edu

***Abstract*—This lab implements and evaluates the k-Nearest Neighbors (k-NN) classification algorithm on the supplied Uniswap V3 swap dataset. The dataset contains 100,000 swap records. For the binary classification experiments, the fee tiers 100 and 3000 were selected as the two classes. The implementation includes categorical encoding, missing-value handling, distance calculation, three configurable sorting algorithms, nearest-neighbor selection, majority voting, distance-weighted voting, and a modular `fit()`, `predict()`, and `score()` interface. The custom implementation is compared with Scikit-learn's `KNeighborsClassifier` over multiple values of k.**

***Index Terms*—k-nearest neighbors, classification, Uniswap V3, frequency encoding, distance weighting, data preprocessing, Scikit-learn**

I. Introduction

The most-used version is Uniswap V3, an automated market maker (AMM) on Ethereum, and its concentrated-liquidity structure has made it a key venue for token price discovery across the broader DeFi ecosystem. Every swap it processes is settled on-chain, so, unlike a centralized exchange's internal order book, the full history of trades, pool balances, and counterparties is public. This transparency is often assumed to make manipulation easier to police than in traditional or centralized crypto markets. In practice it has also made wash trading harder to observe, because a manipulator can spread trades across sender, recipient, and origin addresses that all trace back to the same actor, and the swap simply looks like ordinary flow through a liquidity pool rather than a matched trade between two counterparties.

Wash trading, executing offsetting trades that create the appearance of volume without any real change in the trader's market position, has been documented at scale on both centralized exchanges [5], [10] and decentralized exchanges [16], [8]. On AMM venues specifically, Gan et al. [8] recently estimated that more than a million abnormal transactions worth over $27 billion have passed through Uniswap V2 and V3 pools. The consequences mirror those documented in traditional markets: inflated volume rankings, misleading liquidity signals to prospective liquidity providers and traders, and, on protocols that reward activity with token incentives, the direct extraction of liquidity-mining or trading rewards.

Detecting this behavior on Uniswap V3 poses two problems specific to the AMM setting. First, there is no natural counterparty to compare a trade against, since every swap executes against a pool rather than another trader; wash-trading signals have to be reconstructed from patterns across addresses, transactions, and pools rather than from matched buy/sell pairs. Second, and more fundamentally, there is no reliable ground truth: no public dataset labels which Uniswap V3 swaps are manipulative, and the pseudonymity of Ethereum addresses makes manual labeling infeasible at scale. These two constraints rule out standard supervised classification and motivate an unsupervised approach, one that can surface suspicious structure directly from the transaction data itself.

This paper takes that approach. Starting from raw Uniswap V3 swap events, we construct a feature-engineered dataset that captures behavior known to be associated with wash trading in related settings: same-transaction, same-pool round trips in opposite directions; the role each address plays (externally owned account, contract, shared or private address) and how often sender, recipient, and transaction origin coincide; how concentrated a pool's volume is among a small number of origins; the share of a pool's trades that reverse a recent trade; and the burstiness of trading activity over time. We then apply unsupervised machine learning to this feature set to flag pools, addresses, and transaction clusters that depart from the trading patterns of the surrounding market, without relying on any labeled examples of wash trading.

The remainder of this paper is organized as follows. Section 2 reviews prior work on wash trading detection in centralized and decentralized markets, unsupervised and anomaly-detection methods applied to blockchain transaction data, and the market microstructure of Uniswap V3 that motivates our feature choices. Section 3 describes the dataset and feature engineering pipeline. Section 4 presents the unsupervised models used to flag suspicious activity. Section 5 reports and discusses the results, and Section 6 concludes.

II. Literature Survey

Research relevant to this paper falls into three strands: work that detects wash trading in cryptocurrency markets, work that applies unsupervised or anomaly-detection methods to blockchain transaction data more broadly, and work on the market microstructure of Uniswap and other automated market makers that shapes which signals are informative on-chain.

A. Wash Trading in Centralized and Decentralized Cryptocurrency Markets

Wash trading was first formalized as a detection problem in traditional financial markets by Cao, Li, Coleman, Belatreche, and McGinnity [2], [3], who modeled trades as a directed graph and searched for subsets of trades that returned every participant to their original position; their exact algorithm

has exponential complexity, which later crypto-focused work had to work around at blockchain scale. Cong, Li, Tang, and Yang [5] brought this question to centralized cryptocurrency exchanges, testing 29 platforms for statistical fingerprints of fabricated volume, deviations in first-significant-digit distributions, trade-size rounding, and tail behavior, and found that unregulated exchanges wash-traded an average of over 70% of their reported volume, while regulated exchanges did not. Le Pennec, Fiedler, and Ante [10] cross-validated reported volume against independent signals such as web traffic and wallet balances across twelve exchanges, finding suspicious volume exceeding 90% at several of them and corroborating the exchange-level statistical tests with an outside benchmark.

Because centralized-exchange studies can only observe reported trades and not account-level identities, a separate line of work studies wash trading directly on-chain. Victor and Weintraud [16] analyzed two early order-book-based decentralized exchanges, IDEX and EtherDelta, by building a directed token-trade graph, identifying strongly connected components as candidate colluding account sets, and then matching trade volumes within those sets to find subsets that left every account's position unchanged. They found over $159 million in wash-traded volume and noted that more than 30% of tokens on both exchanges were affected, but explicitly left automated-market-maker exchanges such as Uniswap as open future work, since AMM traders swap against a pool rather than against each other.

Gan, Wang, Xue, and Lin [8] address exactly that gap: rather than looking for cycles between trading accounts, they link addresses that share the same funding source for transaction fees and flag pairs of linked addresses that buy and sell the same asset in close succession while their combined holdings stay nearly constant. Applied to 98,945 pools on Uniswap V2 and V3, their method reported 95.9% recall and flagged over a million abnormal transactions worth roughly $27.51 billion, making it, to our knowledge, the closest prior study to the Uniswap V3 setting used in this paper.

Wash trading detection has also been extended to NFT markets, which share Ethereum's pseudonymity and public ledger with Uniswap. Falk, Tsoukalas, and Zhang [7] built trade-level labels for NFT marketplaces from on-chain filters, matching wallets, back-and-forth trades, and repeated purchases of the same asset, and then trained tree-based and deep-learning classifiers on the resulting labeled data, achieving accurate trade-level classification once ground truth existed. Chen, Chen, Yu, Luo, and Wang [4] conducted a large-scale empirical study of NFT wash trading using similar graph-based heuristics. Both lines of NFT work illustrate a recurring pattern in this literature: once labels can be constructed from behavioral heuristics, supervised classifiers perform well, but the labels themselves still come from hand-built rules rather than a learned representation of normal trading behavior.

B. Unsupervised and Anomaly-Detection Methods for Blockchain Transaction Data

A parallel body of work applies unsupervised learning to blockchain transaction data for anomaly and fraud detection generally, motivated by the same problem this paper faces: labeled examples of illicit activity are scarce or nonexistent. Pham and Lee [15] were among the first to apply unsupervised methods, k-means clustering, Mahalanobis distance, and one-class SVM, to Bitcoin user and transaction graphs, using distance from cluster centroids to flag outlying accounts. Liu, Jiang, Liu, and Tse [11], in a survey of knowledge discovery in cryptocurrency transactions, catalog the unsupervised techniques used for this kind of anomaly detection across the literature, including k-means, kd-tree methods, one-class SVM, Isolation Forest, Gaussian mixture models, and RolX-style role factorization, and note that all of them arose from the same shortage of reliable ground truth that supervised classifiers would otherwise need.

Later work has combined or extended these building blocks. Nadella, Meduri, Gonaygunta, Satish, and Vadakkethil Somanathan Pillai [12] applied K-means clustering together with Isolation Forest to blockchain transaction data, arguing that no single unsupervised method captures every anomaly type and that pairing a partition-based clustering step with a density- or isolation-based detector improves coverage. Patel, Reiner, Stilwell, Wahbeh, and Seetan [13] found that both Isolation Forest and DBSCAN struggle with contextual outliers on Bitcoin transactions, points that are only anomalous relative to a specific subgroup of transactions, and proposed pairing k-means clustering with a within-cluster z-score to address this. Gu, Lin, and Wu [9] took a different route for cryptocurrency exchanges, training a deep-learning model to predict expected transaction amounts from correlated exchange factors and flagging transactions whose observed amount deviates from that prediction, an approach that needs no labeled anomalies. Separately, Patel, Pan, and Rajasegarar [14] moved from flat feature vectors to graph representations, showing that a one-class graph neural network for Ethereum transaction networks outperforms traditional non-graph anomaly detectors such as Isolation Forest, which they attribute to Isolation Forest's inability to capture relationships between accounts. Taken together, this literature establishes that unsupervised clustering and outlier-scoring methods are an accepted way to surface suspicious blockchain activity when labels are unavailable, but almost all of it is applied to wallet-level or account-graph features rather than to swap-level features engineered from an AMM's own trading mechanics.

C. Uniswap V3 Mechanics and DEX Market Microstructure

The signals available for wash-trading detection on Uniswap V3 are shaped by how the protocol itself works. Unlike Uniswap V2's uniform liquidity across the full price range, Uniswap V3 lets liquidity providers concentrate capital within chosen price ranges and lets pools coexist at multiple fee tiers for the same token pair (Adams, Zinsmeister, Salem, Keefer, & Robinson, 2021). This is why pool-level features such as fee

tier, volume concentration, and reversal share are meaningful units of analysis rather than the token pair alone. Because all pending and confirmed transactions are visible on-chain, Uniswap and other DEXs are also exposed to strategies that exploit that visibility for profit rather than for wash trading specifically: Daian et al. [6] documented systematic front-running and transaction-reordering ("miner extractable value") on decentralized exchanges, showing that transparency which should make manipulation easier to detect can simultaneously be exploited by other actors. Xu, Paruch, Cousaert, and Feng [17] provide a broader systematization of AMM-based decentralized exchanges, covering pricing mechanisms, liquidity provision, and the security and manipulation risks that differ from order-book exchanges, context that motivates treating AMM swap data as its own object of study rather than reusing detection methods built for order books.

D. Positioning of This Work

Across these three strands, DEX-specific wash-trading studies rely on hand-built heuristics, strongly-connected-component search with volume matching (Victor & Weintraud, 2021), or ETH-fee-based entity linking (Gan et al., 2024), applied to Uniswap and its predecessors, rather than on a data-driven unsupervised model. Conversely, the unsupervised and anomaly-detection literature reviewed above (Pham & Lee, 2016; Liu et al., 2020) is well developed for blockchain data in general but has largely been applied to wallet balances and account-level transaction graphs, not to the pool-, address-, and transaction-level features that are specific to an AMM's swap mechanics. The supervised NFT wash-trading classifiers (Falk et al., 2026) show that machine learning performs well once labels exist, but depend on those labels being constructed by the same kind of heuristic filters this paper aims to avoid. This paper sits at the intersection of these strands: it applies unsupervised machine learning, in the spirit of the general blockchain anomaly-detection literature, to a feature set engineered specifically for Uniswap V3 swap data, in order to flag wash-trading-like behavior without relying on hand-labeled examples or hard-coded heuristic thresholds.

III. Methodology

A. Experimental Dataset and Binary Target

There are 20 columns and 100,000 Uniswap V3 swap records supplied in the file. Here are four values of fee tiers in the full data set. The experiments deliberately limit the target to fee levels of 100 and 3000 for making the laboratory task a binary classification problem. For fee tier 100, the raw class counts are 19,145 and for fee tier 3000, the raw class counts are 32,899 and the rest of the class counts are either 500 or 10,000, and omitted from the binary experiment.

To be able to make the nearest neighbour calculations by hand without losing any balance in the classes, 600 observations are sampled from each selected class, using the same random number seed of `random_state=42` for all of them. The experimental data set is therefore comprised of 1200 records and has an exactly balanced class distribution. The class proportions are equal prior to split so that there will be 420 training observations and 180 testing observations per class after the stratified split. This gives a total of 840 training samples and 360 testing samples.

TABLE I
Dataset Construction and Experimental Split

Quantity	Value
Raw swap records	100,000
Raw fee tier 100	19,145
Raw fee tier 500	22,828
Raw fee tier 3000	32,899
Raw fee tier 10,000	25,128
Selected class 100	600
Selected class 3000	600
Balanced experiment size	1,200
Training samples	840
Testing samples	360
Features after preprocessing	11

B. End-to-End Processing Pipeline

The complete processing pipeline follows the sequence used in the experiment. The raw file is first loaded and filtered to the two selected target values. The balanced sample is then shuffled using the fixed random state. The target column is separated from the input matrix before any feature transformation is performed.

The training and testing records are generated before the categorical frequency maps and standardization parameters are fitted. This ordering is important because it ensures that the category frequencies and scaling parameters are learned from training data and then applied to the test data rather than being re-estimated independently on the test set.

C. Feature Representation

The final feature vector contains eleven dimensions:

$$x = [t, a_0, a_1, a_{\mathrm{USD}}, d_0, d_1, b, g_u, g_p, f_0, f_1], \tag{1}$$

where t is timestamp, a_0 and a_1 are the two swap amounts, a_{USD} is the USD-denominated amount, d_0 and d_1 are token decimals, b is block number, g_u is gas used, g_p is gas price, and f_0, f_1 are the frequency-encoded token symbols.

The feature selection intentionally excludes identifiers and address/hash fields such as `swap_id`, sender, recipient, origin, token addresses, pool address, and transaction hash. The target `fee_tier` is also removed. This produces a compact numerical representation that can be passed directly to the Euclidean distance function.

D. Categorical Missing-Value Handling and Frequency Encoding

The two token-symbol columns are categorical. Missing categorical entries are replaced with the mode computed in the corresponding data frame. Each categorical value is then mapped to the relative frequency estimated from the training subset. For a category c in feature column j, the encoded value is

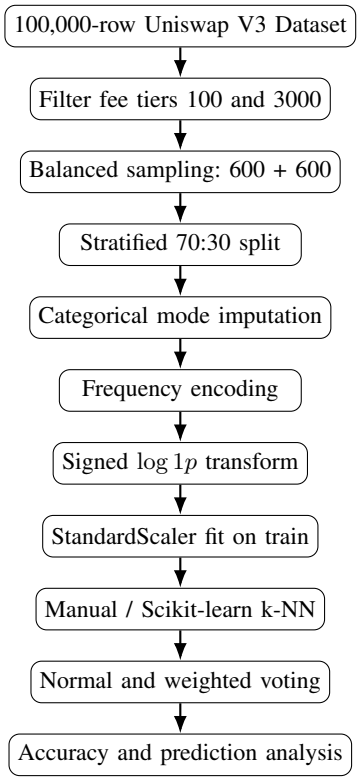


Fig. 1. End-to-end workflow used for the Lab 5 k-NN experiments.

$$F_j(c) = \frac{N_j(c)}{N_j}, \tag{2}$$

where $N_j(c)$ is the number of training observations containing category c and N_j is the number of training observations in that column.

One symbolic vector is converted to a single numeric feature using frequency encoding, but the information in the symbolic vector about the frequency of the token symbol in the training data is retained. The symbols in the test set are not seen and mapped to zero. This is especially applicable for this dataset as there are many symbols that are present in the raw file that cannot be represented in the final k-NN representation except numerically.

E. Numerical Transformation and Standardization

The transaction-related variables have substantially different magnitudes.implementation thus performs a signed logarithmic transformation on `amount0`, `amount1`, `amountUSD`, `gas_used` and `gas_price`:

$$x' = \text{sign}(x)\log(1 + |x|). \tag{3}$$

This change will decrease the influence of very high positive or negative numbers, but will not alter the sign of the original number.

The feature matrix is transformed and categorized and then standardized with the training data:

TABLE II
SORTING METHODS USED IN THE MANUAL IMPLEMENTATION

Algorithm	**Time**	**Role in Lab**	**Measured Demo Accuracy**
Bubble	$O(n^2)$	Required A1 sorter	92.50%
Insertion	$O(n^2)$	Required A1 sorter	92.50%
Selection	$O(n^2)$	Required A1 sorter	92.50%
Merge	$O(n \log n)$	Full manual evaluation	88.61%

$$z_j = \frac{x_j - \mu_j}{\sigma_j}, \tag{4}$$

The mean and standard deviation of the training-set for feature j are μ_j and σ_j. The fitted scaler is then applied to test data without re-fitting the scaler onto the test data. The standardisation is of special importance for k-NN as the classification result is directly based on the distances measured in feature space.

F. A1 – Modular k-NN

Each of the manual implementation is independent modules which are used for calculating distance, sorting, selecting neighbor and voting class. The Euclidean distance metric was the primary one used. Given a query point x and a training point x_i the Euclidean distance is

$$d(x, x_i) = \sqrt{\sum_j (x_j - x_{ij})^2}. \tag{5}$$

The nearest neighbor list is a list of tuples that have the distance, training sample number, and class label. The index is kept to ensure that cases are sorted in a deterministic manner on equal distance. The first k entries in the sorted list constitute the neighborhood with which to make the prediction.

G. Sorting Algorithms and Complexity

The lab needs to have three different elementary sorting algorithms: bubble sort, insertion sort, and selection sort. Both methods were done explicitly, and the correctness was checked with the sample list $[4, 1, 3, 2, 5]$ for which both implementations returned the list $[1, 2, 3, 4, 5]$.

In the A1 demonstration each sorting method was additionally applied to a 40 sample test subset on 250 training samples, $k = 3$. The measured accuracies of all three were the same, 0.9250. However, the implementation was based on the $O(n^2)$ technique of merge sort, which can be found in the sorting module, otherwise the whole experiment with a full evaluation would be too costly if the $O(n^2)$ technique was used repeatedly.

The total number of pair-wise distance evaluations in the full test set is 302,400, with one prediction pass done per test observation over 360 test observations on 840 training samples per observation. For each query used in the tested k values, the same sorted-neighbor list is used for both A8 and A9, thus the same distance calculations are useful for all values of k from $k = 1$ to $k = 15$.

H. Majority Voting

For ordinary k-NN, the predicted class is the class occurring most frequently among the selected k neighbors. When multiple classes obtain the same vote count, the implementation resolves the tie using the smaller class label. Because the selected target labels are 100 and 3000, this rule makes tie behavior deterministic.

The use of odd k values from 1 to 15 also reduces the frequency of equal-vote cases in this binary classification setting. Nevertheless, the explicit tie rule remains part of the implementation and makes the classifier deterministic for all inputs.

I. A2 – Distance-Weighted k-NN

The weighted version gives larger influence to nearer observations. The implementation computes

$$w_i = \frac{1}{\max(d_i, \epsilon)}, \tag{6}$$

where $\epsilon = 10^{-12}$ is a zero-distance safeguard. The total weight for each class is the sum of the weights of its selected neighbors, and the class with the larger total weight is predicted.

Compared with ordinary majority voting, this approach allows a very close training sample to contribute more strongly than a more distant sample within the same neighborhood. Thus, the value of k controls the number of candidates considered, while the inverse-distance rule controls how strongly their distances influence the final decision.

J. A3 – Train/Test Split

The data is divided into training and testing sets using Scikit-learn's `train_test_split` with a 70:30 split and stratification. The measured shapes are $(840, 11)$ for the training feature matrix and $(360, 11)$ for the testing feature matrix. Each split contains 420 training and 180 testing samples from each class.

The fixed random state of 42 makes the sample selection and split reproducible. The resulting accuracy values in this report therefore correspond to the exact measured experiment stored with the lab outputs.

K. A4 and A5 – Scikit-learn k-NN and Score

A Scikit-learn `KNeighborsClassifier` with $k = 3$ is trained on the training data. The `score()` method is used to measure both training and test accuracy. The recorded training accuracy is 0.9226, corresponding to 775 correct classifications out of 840 training observations. The recorded test accuracy is 0.8861, corresponding to 319 correct classifications out of 360 test observations.

The Scikit-learn implementation therefore provides a package-level reference against which the manual implementation can be checked. The same preprocessing, train/test split, Euclidean metric, and neighborhood size are used in the comparison.

L. A6 – Prediction

The `predict()` method is used to obtain class predictions for the test samples. For the first 20 test observations, 19 predicted labels match the recorded actual labels, giving 95% accuracy on this small displayed subset. This value is treated only as a local inspection of the printed output; the full 360-sample test result is the primary evaluation metric.

M. A7 – fit, predict and score

The custom implementation is encapsulated in a `ManualKNN` class exposing the same high-level operations expected from a conventional machine-learning estimator: `fit()`, `predict()`, and `score()`.

The recorded A7 output shows a demo training score of 0.9250 on the first 120 training observations and a test score of 0.8861 on the full testing set. The package therefore demonstrates both modular functions and a reusable estimator-style interface.

N. A8 – Comparison for Different Values of k

Normal k-NN was evaluated for the odd values $k = 1, 3, 5, \ldots, 15$. For every value of k, the manual implementation and Scikit-learn were run on the same preprocessed training and testing matrices. The comparison therefore isolates implementation differences while keeping the data and evaluation protocol fixed.

O. A9 – Weighted Comparison

The same range of k values was evaluated using inverse-distance weighted voting. The manual implementation was compared against Scikit-learn's `weights='distance'` configuration. The weighted experiment provides a direct comparison between ordinary and distance-sensitive neighborhood aggregation.

IV. Experimental Results

A. A1 – Manual Modular Implementation

The sorting demonstration produced the ordered output $[1, 2, 3, 4, 5]$ for bubble, insertion, and selection sorting. More importantly, all three sorters achieved exactly the same accuracy, 92.50%, on the 40-query demonstration subset with 250 training samples. This confirms that, for the tested subset, changing the ordering mechanism did not alter the selected nearest labels.

The full A1 manual k-NN experiment achieved 0.8861 accuracy at $k = 3$, or 88.61%. On 360 test observations, this represents 319 correct predictions and 41 incorrect predictions.

B. A2 – Weighted k-NN

At $k = 5$, the weighted manual implementation achieved 0.8917 test accuracy. This corresponds to 321 correct predictions out of 360 observations and 39 incorrect predictions. The result already exceeds the ordinary k-NN result at $k = 5$, which is 88.06%.

TABLE III
MEASURED A3–A7 OUTPUTS

Task	Measured result	Value
A3	Training / testing samples	840 / 360
A3	Features	11
A3	Per-class train / test counts	420 / 180
A4	Scikit-learn k	3
A5	Training accuracy	92.26%
A5	Testing accuracy	88.61%
A6	First-20 prediction accuracy	95.00%
A7	Demo training score	92.50%
A7	Testing score	88.61%

TABLE IV
NORMAL K-NN TEST ACCURACY FOR DIFFERENT VALUES OF k

k	Manual (%)	Scikit-learn (%)	Difference
1	86.67	86.67	0.00
3	88.61	88.61	0.00
5	88.06	88.06	0.00
7	86.67	86.67	0.00
9	86.67	86.67	0.00
11	86.39	86.39	0.00
13	86.39	86.39	0.00
15	87.22	87.22	0.00

C. A3–A7 Execution Summary

Table III consolidates the directly measured console outputs for the main individual lab tasks.

D. A8 – Normal k-NN Across k

Table IV reports all measured test accuracies. The manual and Scikit-learn values agree at every tested k. The maximum observed accuracy is 88.61% at $k = 3$, followed by 88.06% at $k = 5$. The minimum in the tested range is 86.39% at $k = 11$ and $k = 13$.

The normal-k-NN curve varies by only 2.22 percentage points across the tested interval, from 86.39% to 88.61%. The sharpest change is the increase from 86.67% at $k = 1$ to 88.61% at $k = 3$, followed by a gradual decline to 86.39% at $k = 11$ and $k = 13$. At $k = 15$, accuracy rises again to 87.22%.

E. A9 – Weighted k-NN Across k

The weighted results are shown in Table V and Fig. 3. Manual and Scikit-learn results again match at every value. The highest observed weighted accuracy is 89.17%, occurring at both $k = 3$ and $k = 5$.

Relative to the corresponding normal model, weighting increases test accuracy at every tested value except $k = 1$, where both methods are identical by construction. The largest measured gain is 1.39 percentage points at $k = 9$. At $k = 3$ and $k = 5$, weighted accuracy is 89.17%, compared with 88.61% and 88.06%, respectively, for ordinary voting.

F. Confusion Matrix and Per-Class Results

To interpret the aggregate accuracy more closely, the full $k = 3$ predictions were also examined by class. For ordinary k-NN, the confusion matrix contains 162 correctly classified class-100 samples and 157 correctly classified class-3000 samples. There are 18 class-100 samples predicted as 3000 and 23 class-3000 samples predicted as 100.

For ordinary k-NN, class 100 has precision 87.57%, recall 90.00%, and F1-score 88.77%. Class 3000 has precision 89.71%, recall 87.22%, and F1-score 88.45%. The macro-average F1-score is 88.61%.

For weighted k-NN at $k = 3$, class 100 has precision 87.70%, recall 91.11%, and F1-score 89.37%. Class 3000 has precision 90.75%, recall 87.22%, and F1-score 88.95%. The macro-average F1-score is 89.16%.

These values show that the accuracy difference between the two classes is small, but the error directions are not identical. The weighted configuration reduces false negatives for class 100 from 18 to 16 while leaving the class-3000 row unchanged in this particular test split.

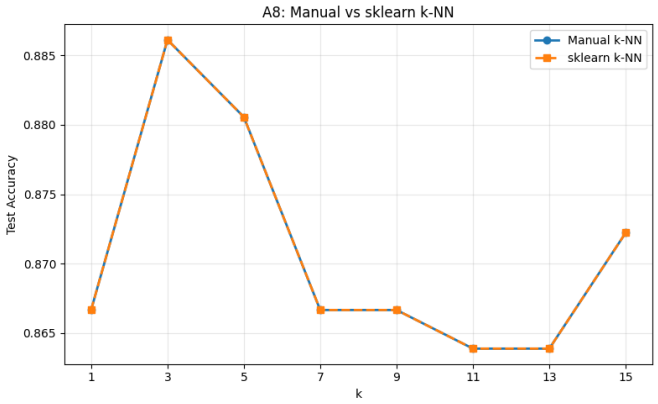


Fig. 2. Manual versus Scikit-learn k-NN test accuracy across the tested values of k. The curves overlap at every point.

TABLE V
WEIGHTED K-NN TEST ACCURACY FOR DIFFERENT VALUES OF k

k	Manual (%)	Scikit-learn (%)	Normal (%)	Gain (pp)
1	86.67	86.67	86.67	0.00
3	89.17	89.17	88.61	0.56
5	89.17	89.17	88.06	1.11
7	87.50	87.50	86.67	0.83
9	88.06	88.06	86.67	1.39
11	86.94	86.94	86.39	0.56
13	87.50	87.50	86.39	1.11
15	88.33	88.33	87.22	1.11

V. OBSERVATIONS AND DISCUSSION

A. Class Separation

The measured test accuracies lie between 86.39% and 89.17% over the tested settings. This indicates that the selected features contain information that allows the two fee-tier classes to be distinguished, although the decision is not perfect. The confusion matrix at $k = 3$ shows 41 errors for ordinary k-NN and 39 errors for weighted k-NN. Both classes therefore have substantial correct classification counts, while a non-trivial

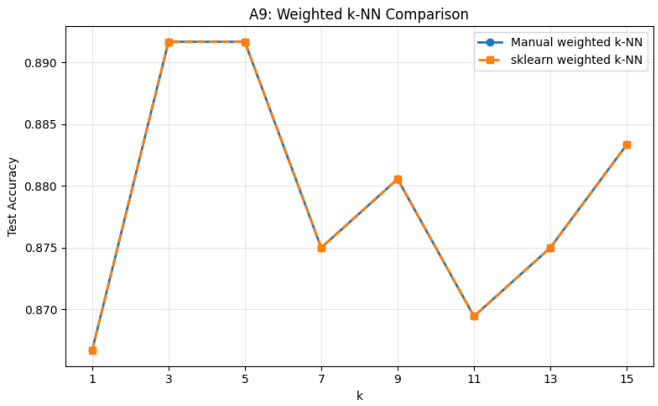


Fig. 3. Weighted manual and Scikit-learn k-NN test accuracy across different values of k.

TABLE VI
CONFUSION MATRICES FOR $k = 3$

	Normal		Weighted	
Actual	100	3000	100	3000
100	162	18	164	16
3000	23	157	23	157

minority of samples remains difficult to distinguish using local feature similarity.

The balanced test set also makes the accuracy interpretation direct: each class contributes exactly 180 test observations. Consequently, an accuracy of 88.61% corresponds to 319 correct decisions out of 360, rather than being dominated by a majority class.

B. Effect of Increasing k

The test accuracy does not increase monotonically with k. For normal k-NN, it rises from 86.67% at $k = 1$ to 88.61% at $k = 3$, decreases to 88.06% at $k = 5$, and then reaches 86.39% at $k = 11$ and $k = 13$ before increasing to 87.22% at $k = 15$.

The weighted curve has a different pattern. It reaches 89.17% at both $k = 3$ and $k = 5$, drops to 87.50% at $k = 7$, rises to 88.06% at $k = 9$, and then fluctuates between 86.94% and 88.33% for the remaining tested values. This confirms that the effect of neighborhood size depends on the vote aggregation rule.

C. Small-k Behavior and Generalization

At $k = 1$, the decision is determined by a single nearest training point. The measured normal and weighted accuracies are both 86.67%. As k increases to 3, the normal test accuracy increases by 1.94 percentage points. The improvement indicates that, for this fixed train/test split, incorporating several local neighbors provides a more stable decision than relying on one observation.

The training accuracy at $k = 3$ is 92.26%, while the test accuracy is 88.61%. The 3.65 percentage-point gap demonstrates

TABLE VII
PER-CLASS CLASSIFICATION MEASURES AT $k = 3$

	Normal			Weighted		
Class	P	R	F1	P	R	F1
100	87.57	90.00	88.77	87.70	91.11	89.37
3000	89.71	87.22	88.45	90.75	87.22	88.95

that performance on the training observations is higher than performance on unseen test observations. The results alone do not establish severe overfitting, but the difference is consistent with the general property that highly local neighborhood rules can be sensitive to the training set.

D. Large-k Behavior and Smoothing

Larger neighborhoods combine more training observations into each decision. In this experiment, the decline from the 88.61% peak at $k = 3$ to 86.39% at $k = 11$ and $k = 13$ shows that increasing the neighborhood size does not automatically improve test accuracy.

A large neighborhood can include observations from both classes even when the query lies near a locally informative region. The resulting vote becomes more global and can smooth away local structure. The present measurements therefore provide an empirical example of the trade-off between local sensitivity and broad neighborhood smoothing.

E. Manual Implementation Versus Scikit-learn

The manual and Scikit-learn implementations produced exactly identical accuracy values for all eight tested values of k in A8. The same equality holds across all eight values in A9. Thus, there is a 0.00 percentage-point discrepancy at every comparison point.

This agreement is important because the manual implementation contains several independently written stages: Euclidean distance calculation, sorting, neighbor truncation, majority voting, and weighted voting. Reproducing the package results under the same configuration provides evidence that these stages are internally consistent for the tested dataset and split.

F. Effect of Distance Weighting

The weighted approach improves the test accuracy relative to ordinary voting at seven of the eight tested k values, while at $k = 1$ the two methods are necessarily identical because only one neighbor participates.

The largest gain is 1.39 percentage points at $k = 9$. At the commonly examined $k = 3$, the gain is 0.56 percentage points, increasing accuracy from 88.61% to 89.17%. At $k = 5$, the gain is 1.11 percentage points, increasing accuracy from 88.06% to 89.17%.

The improvement is modest in absolute terms, but it is consistent across most of the tested settings. The observed result supports the use of the weighted experiment as a meaningful extension of the basic majority-voting classifier rather than as a completely different model.

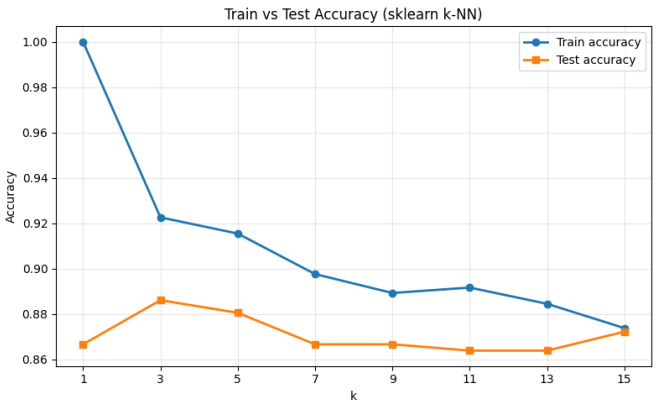


Fig. 4. Training and test accuracy of the Scikit-learn k-NN classifier across the tested values of k.

G. Training Versus Testing Performance

For $k = 3$, the Scikit-learn model obtains 92.26% training accuracy and 88.61% test accuracy. In terms of counts, this is 775 correct classifications among 840 training samples and 319 correct classifications among 360 test samples.

The difference is 3.65 percentage points. Because k-NN is an instance-based method, the training score is naturally influenced by the presence of each training observation within the reference set. The test score provides the more direct measure of how the learned representation and neighborhood rule behave on observations not used to determine the neighborhood for that sample.

Fig. 4 extends this comparison from $k = 3$ to the complete tested range. At $k = 1$ the training accuracy is 100%, because each training observation is its own nearest neighbor, whereas the test accuracy is 86.67%, a gap of 13.33 percentage points. This is the clearest sign of overfitting in the experiment, since the classifier reproduces the training labels exactly but generalizes noticeably worse. As k increases, the training accuracy generally decreases while the test accuracy peaks at 88.61% at $k = 3$, so the gap narrows to 3.65 percentage points at $k = 3$ and to less than one percentage point at $k = 15$. At $k = 15$ the two curves almost coincide at about 87%, which shows that the larger neighborhood no longer fits the training data better than unseen data and that further smoothing does not improve test accuracy. Together, the curves indicate that small k values are prone to overfitting, large k values lose local detail, and $k = 3$ to $k = 5$ provides the best balance between training fit and generalization in this experiment.

H. Interpretation of A6 Predictions

The first 20 printed test predictions contain 19 correct predictions and one mismatch. The displayed outputs therefore illustrate that the model can make consecutive correct decisions across both classes, but also that a locally similar sample can be assigned the wrong fee tier.

This output is useful as a qualitative check of the `predict()` function, whereas the full 360-sample accuracy should be used for quantitative evaluation. The distinction prevents a small displayed sample from being treated as a replacement for the full test result.

I. A7 Estimator Interface

The A7 experiment demonstrates that the manual model can be wrapped in a conventional estimator interface. The `fit()` method stores the training feature matrix and labels, `predict()` delegates classification to the manual prediction routine, and `score()` computes the mean fraction of correct predictions.

This modularization separates implementation from experiment logic. As a result, A8 and A9 can reuse the same core prediction routines with different values of k and different vote rules. The identical outputs across the modular function version and the class-based version provide an additional consistency check.

J. Computational Considerations

The manual implementation is intentionally explicit for laboratory study. The required bubble, insertion, and selection sort routines each have quadratic worst-case time complexity, so they are appropriate for demonstrating data-structure and algorithm concepts but are not used for the full 840-sample prediction pass. Merge sort is used in the full manual experiment to provide an $O(n \log n)$ ordering step.

For every query in a full prediction pass, the algorithm computes distances to all 840 training observations. With 360 test observations, this gives 302,400 distance evaluations. A8 and A9 then reuse the sorted neighbor sequence for multiple k values, so changing k does not require another set of distance computations inside the manual `predict_many_k` routine. No execution-time benchmark is claimed here because the experiment reports accuracies and console results rather than a controlled timing experiment.

K. Answers to the Laboratory Questions

The experiments answer the required analytical questions directly. The two classes are not perfectly separated, as shown by test errors at every value of k. Test accuracy increases from $k = 1$ to $k = 3$ and then generally fluctuates or decreases. The difference between training and test accuracy at $k = 3$ provides evidence that the model does not behave identically on seen and unseen observations.

The manual implementation closely matches Scikit-learn, with zero measured accuracy difference for every tested normal and weighted setting. Distance weighting changes the results in this experiment, producing a maximum of 89.17% compared with a maximum of 88.61% for ordinary k-NN. The largest reduction in normal test accuracy occurs around $k = 11$ and $k = 13$, where it falls to 86.39%, illustrating the loss of local specificity as the neighborhood expands. Across the tested settings, the measured results show that $k = 3$ is a high-performing small neighborhood for ordinary k-NN, while weighted voting also attains 89.17% at $k = 5$.

TABLE VIII
QUANTITATIVE SUMMARY OF THE MAIN EXPERIMENTAL FINDINGS

Measure	Value	Interpretation
Normal best test accuracy	88.61%	$k = 3$
Weighted best test accuracy	89.17%	$k = 3$ and $k = 5$
Weighted gain at $k = 3$	0.56 pp	vs. normal $k = 3$
Weighted gain at $k = 5$	1.11 pp	vs. normal $k = 5$
Manual–Scikit-learn difference	0.00 pp	all tested k
Normal accuracy range	86.39–88.61%	$k = 1$ to 15
Weighted accuracy range	86.67–89.17%	$k = 1$ to 15
Normal $k = 3$ errors	41 / 360	11.39%
Weighted $k = 3$ errors	39 / 360	10.83%

L. Reproducibility and Experimental Limitations

The experiments are deterministic at the data-selection level because the balanced class sampling and train-test split both use `random_state=42`. The binary dataset contains exactly 600 samples from each selected fee tier before splitting, and stratification preserves the 50:50 class ratio in both training and testing sets. These choices make the numerical values in this report reproducible when the same input file and software configuration are used.

The experiments also have several scope limitations. First, the evaluation uses one fixed 70:30 train-test split rather than repeated cross-validation. The reported accuracies therefore describe this particular split and should not be interpreted as an estimate of variability across multiple samples. Second, the manual A1 sorting demonstration uses a reduced subset for the quadratic algorithms, while the full experiment uses merge sort. Consequently, the 92.50% sorting-demo accuracy and the 88.61% full-test accuracy are results from different computational subsets and serve different purposes.

Third, the selected target is `fee_tier`, and the experiment is specifically a binary distinction between 100 and 3000. The results should therefore be interpreted as evidence about this laboratory classification formulation rather than as a general statement about all Uniswap V3 transactions or all fee tiers. Fourth, frequency encoding is learned from the training subset and maps unseen test categories to zero. This is reproducible and avoids fitting the mapping on the test set, but it also means that the numerical meaning of an encoded symbol depends on the training population.

Finally, the stored outputs do not contain a controlled runtime benchmark across algorithms and implementations. The computational discussion is therefore based on operation counts and asymptotic sorting complexity rather than measured execution time. Future evaluations could repeat the experiment across multiple random seeds, report confidence intervals, and compare additional distance metrics while preserving the same preprocessing protocol.

VI. CONCLUSION

This lab implemented and evaluated a modular k-Nearest Neighbors classifier for a binary classification problem derived from the supplied Uniswap V3 swap dataset. The workflow included class filtering, balanced sampling, stratified splitting, categorical mode imputation, frequency encoding, signed logarithmic transformation, standardization, Euclidean distance calculation, explicit sorting algorithms, nearest-neighbor selection, majority voting, inverse-distance weighting, and a reusable `fit()`, `predict()`, and `score()` interface.

The measured experiment used 1,200 balanced observations, with 840 training and 360 testing samples and 11 final numerical features. At $k = 3$, ordinary Scikit-learn k-NN achieved 92.26% training accuracy and 88.61% test accuracy. The manual implementation reproduced the same 88.61% test result. Across $k = 1$ through $k = 15$ at odd values, manual and Scikit-learn test accuracies were identical at every point.

The weighted experiment produced a maximum measured test accuracy of 89.17% at both $k = 3$ and $k = 5$. At $k = 3$, this is a 0.56 percentage-point increase over ordinary k-NN; at $k = 5$, the increase is 1.11 percentage points. The weighted and package implementations again matched exactly across the complete tested range.

Overall, the results demonstrate that preprocessing and neighborhood design materially affect a distance-based classifier. The fixed split and reproducible pipeline make the reported results directly traceable to the supplied lab outputs, while the agreement between the manual and Scikit-learn implementations provides a quantitative verification of the custom k-NN logic.

VII. REFERENCES

REFERENCES

[1] T. M. Cover and P. E. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967, doi: 10.1109/TIT.1967.1053964.

[2] N. S. Altman, "An introduction to kernel and nearest-neighbor nonparametric regression," *The American Statistician*, vol. 46, no. 3, pp. 175–185, 1992.

[3] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, E. Duchesnay, and M. Brucher, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[4] H. Adams, N. Zinsmeister, M. Salem, R. Keefer, and D. Robinson, "Uniswap v3 Core," Uniswap Labs, 2021.

[5] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.